Unidad 2. Herencia, polimorfismo e interfaces

Programación Orientada a Objetos · Facultad de Ingeniería · Notas de clase para uso académico

1. Herencia: la relación "es un"

La herencia permite definir una clase a partir de otra más general. Un Docente y un Estudiante son Personas: comparten nombre y documento, pero cada uno agrega lo suyo. La subclase hereda atributos y métodos de la superclase y puede añadir nuevos o redefinir los existentes.

La prueba para saber si la herencia tiene sentido es leer la frase "B es un A" en voz alta. "Un Carro es un Vehículo" funciona; "un Carro es un Motor" no: el carro tiene un motor, y eso es composición, no herencia.

2. Sobrescritura y polimorfismo

Polimorfismo significa que una misma llamada produce comportamientos distintos según el objeto real que la recibe. El código cliente trabaja con el tipo general y no necesita preguntar por el tipo concreto con cadenas de if.

```
abstract class Figura {
    abstract double area();
}
class Circulo extends Figura {
    private final double r;
    Circulo(double r) { this.r = r; }
    @Override double area() { return Math.PI * r * r; }
}
class Rectangulo extends Figura {
    private final double b, h;
    Rectangulo(double b, double h) { this.b = b; this.h = h; }
    @Override double area() { return b * h; }
}

double total = 0;
for (Figura f : figuras) total += f.area();   // cada figura sabe calcular su área
```

3. Clases abstractas e interfaces

- Una clase abstracta no se puede instanciar; sirve para compartir código y obligar a las subclases a completar los métodos abstractos.
- Una interfaz define solo un contrato (qué se puede hacer). Una clase puede implementar varias interfaces, pero en Java solo extiende una clase.
- Usa interfaces para capacidades (Comparable, Exportable) y clases abstractas para familias con código común.

4. Composición sobre herencia

Las jerarquías profundas se vuelven rígidas: un cambio en la superclase afecta a todas las subclases. Muchas veces es mejor que un objeto tenga otro y le delegue el trabajo. Esta idea es la base de varios patrones de diseño que se estudian más adelante, como

Strategy y Decorator.

5. Ejercicios de práctica

1. Diseña una jerarquía para empleados de una universidad (docente de planta, catedrático, administrativo) con un método calcularPago(). ¿Qué va en la superclase?
2. Reescribe un bloque if/else que calcule áreas según un String tipo usando polimorfismo.
3. ¿Cuándo usarías una interfaz en lugar de una clase abstracta? Da un ejemplo.
4. Explica por qué Pato extends Avion sería un mal diseño aunque ambos "vuelen".

Fuentes consultadas

• Oracle. The Java Tutorials: Inheritance and Interfaces. docs.oracle.com/javase/tutorial/java/IandI
• Gamma, Helm, Johnson y Vlissides (1994). Design Patterns. Addison-Wesley. Cap. 1 (composición vs. herencia).

Texto original redactado para el curso a partir de las fuentes citadas.